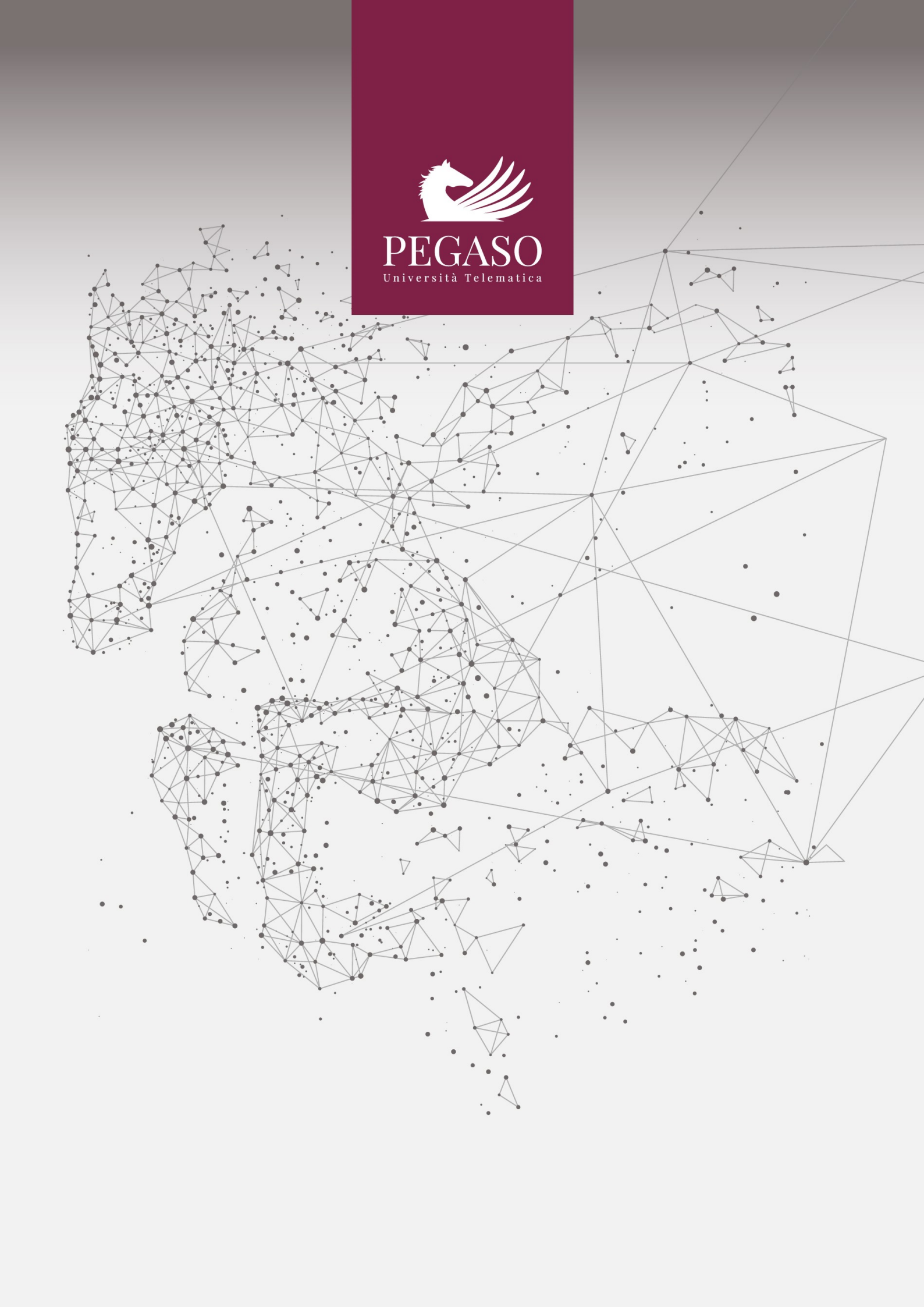

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3
2. STORIA DELL'INGEGNERIA DEL SOFTWARE 5
3. MITI E CONSIDERAZIONI SULLO SVILUPPO E PROGETTAZIONE DEL SOFTWARE 11
BIBLIOGRAFIA 15

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

L'ingegneria del software `e l'insieme delle teorie, dei metodi e delle tecniche

che si usano nello sviluppo industriale del software. Possiamo iniziarne lo studio considerando le seguenti definizioni:

- software

"1-L'insieme o una parte dei programmi, delle procedure, delle regole e della relativa documentazione di un sistema di elaborazione dell'informazione. – 2. Programmi, procedure, ed eventuale documentazione e dati relativi all'operazione di un sistema di calcolo. – 3. Programma o insieme di programmi usati per l'operazione di un calcolatore".

- ingegneria del software

"1. L'applicazione di conoscenze scientifiche e tecnologiche, metodi ed esperienza al progetto, l'implementazione, il collaudo e la documentazione del software. 2. L'applicazione di un approccio sistematico, disciplinato, quantificabile, allo sviluppo, all'operazione ed alla manutenzione del software, cioè l'applicazione dell'ingegneria al software".

"disciplina tecnologica e manageriale che riguarda la produzione sistematica e la manutenzione dei prodotti software, . . . sviluppati e modificati entro i tempi e i costi preventivati".

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

E particolarmente importante, nelle due definizioni di "ingegneria del software", il concetto di sistematicità. La necessità di sottolineare l'importanza di questo concetto, che è dato per scontato nelle altre discipline ingegneristiche, può essere meglio compresa se consideriamo la storia dell'informatica, schematizzata nelle tre fasi di arte, artigianato, e industria.

La prima fase, quella delle origini, è caratterizzata dal fatto che i produttori e gli utenti del software condividevano una formazione scientifica, per cui i programmatori avevano una certa familiarità con i problemi degli utenti, e questi ultimi erano in grado di capire il funzionamento dei calcolatori e quindi le esigenze dei programmatori. Anzi, accadeva spesso che utente e programmatore fossero la stessa persona. Un altro aspetto caratteristico di quel periodo è il fatto che spesso un'applicazione venisse prodotta ad hoc per un particolare problema e venisse abbandonata una volta soddisfatta quella particolare necessità.

La produzione del software in quel periodo era quindi paragonabile ad un'arte, nel senso che era dominata dall'inventività individuale e da un'organizzazione del lavoro molto lasca (tralasciamo qui altri aspetti, pur importanti, legati alle tecniche di programmazione ed agli strumenti di sviluppo, in quel periodo assai limitati).

In una fase successiva, in seguito allo sviluppo delle applicazioni informatiche, estese dal campo scientifico a quello commerciale e amministrativo, i nuovi utenti del software hanno formazioni culturali diverse da quella dei programmatori.

Questi ultimi devono quindi imparare ad affrontare problemi di tipo nuovo, ed a comunicare efficacemente con persone che non solo non hanno le conoscenze necessarie a capire gli aspetti tecnici della programmazione, ma non hanno neppure alcuna motivazione pratica per interessarsene.

Inoltre, le applicazioni richieste hanno un peso economico sempre maggio

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Storia dell'Ingegneria del Software

L'ingegneria del software nasce ufficialmente nel 1968 in occasione della conferenza NATO tenutasi a Garmisch, in Germania. La conferenza registra la partecipazione di numerosi studiosi convenuti a discutere quella che è definita la "crisi del software".

Molti grandi progetti software realizzati in quegli anni per risolvere importanti e complesse problematiche scientifiche e applicative falliscono miseramente non riuscendo a sviluppare software affidabile e rispondente ai requisiti. Inoltre, i tempi di consegna non sono mai rispettati e i costi aumentano a dismisura.

La problematica, conosciuta come "crisi del software", affonda le sue radici nell'inesistenza di una pratica ingegneristica consolidata, con basi scientifiche e metodologicamente definite. Si decide allora di avviare una fase di studio e ricerca per creare i fondamenti di una scienza ingegneristica, battezzata "Software Engineering".

L'anno successivo (1969), una seconda conferenza NATO si tiene a Roma, presso l'American Hotel.

Gli anni pioneristici: 1950 – 1965

- In questi anni il software è sviluppato principalmente per singoli clienti; è di tipo batch e deve risolvere problemi spesso molto complessi.

Gli informatici sono pochi e provenienti da altre discipline ingegneristiche o scientifiche.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Le competenze informatiche si riducono alla conoscenza dei linguaggi di programmazione (linguaggio macchina). Le metodologie di sviluppo sono totalmente assenti e le competenze acquisite sul campo.

I progetti vedono il coinvolgimento di pochi programmatori (spesso uno solo) che sviluppano il codice senza produrre alcuna documentazione.

Lo sviluppo del software si riduce dunque alla mera programmazione. Le uniche discipline applicate sono quelle degli algoritmi, delle strutture dati e dei linguaggi di programmazione.

Due anni importanti: 1968 - 1969

- Le date corrispondono alle due Conferenze NATO tenutesi, come già detto, a Garmisch e a Roma (vedi NATO Conference 1968 e NATO Conference 1969).

La nascita ufficiale dell'Ingegneria del software avvia importanti studi e attività di ricerca nel campo delle metodologie, dei metodi e delle tecniche per il software. Si definisce una nuova disciplina su base teorica e scientifica portando l'Ingegneria del software alla stregua degli altri rami ingegneristici.

La ricerca si sviluppa fondamentalmente su tre direttrici: una linea di ricerca approfondisce i temi legati agli aspetti organizzativi del software definendo ruoli, responsabilità e competenze della professione; un'altra linea si concentra sulle metodologie software (cicli di vita, processi, metodi, tecniche, metriche ecc.); la terza linea si concentrata sulle tecnologie (sistemi operativi, sistemi gestionali, linguaggi di programmazione ecc.).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il periodo che segue la nascita dell'Ingegneria del software è forse tra i più ricchi di personaggi: ci sono tutti quelli che hanno fatto la storia della disciplina. Gli articoli e i libri pubblicati in questi anni costituiscono le basi della disciplina e sono ancora attuali. Per citarne solo alcuni fra i più importanti:

Charles Bachman, The Programmer as Navigator (1973).

Laszlo Belady, Performance Modeling by Peter Denning (1981).

Gli anni della maturità: 1970 - 1990

- In questo periodo si assiste allo sviluppo di software per più utenti/clienti. Le funzioni batch sono affiancate da transazioni on-line e da funzioni real-time. Nascono i "pacchetti software" che offrono soluzioni preconfezionate rivolte a più utenti con la stessa necessità.

I problemi non mancano nemmeno adesso: quello della "manutenzione del software" è il problema più importante!

La diffusione del software aumenta in tutti i settori. Le esigenze degli utenti crescono con il crescere dell'utilizzo del software. Le metodologie definite dall'Ingegneria del software non si diffondono con la giusta attenzione. I metodi e le tecniche sono spesso ignorati/e. Le competenze si limitano alle tecnologie e il software prodotto necessita sempre più di manutenzione: spesso un affare per molti produttori di software.

Il test e collaudo richiede grandi energie, materiali ed economiche, e la qualità del software non sempre risulta adeguata alle necessità.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'introduzione di nuove tecnologie (sistemi distribuiti, nuovi sistemi per la gestione dei dati, linguaggi di programmazione evoluti ecc.) e la riduzione dei costi dell'hardware spingono verso una diffusione sempre maggiore del software applicativo.

Si effettuano analisi sulla difettosità del software e si definisce una "teoria della propagazione degli errori nel software".

Con la programmazione ad oggetti (Object-Oriented) nasce l'innovazione che porterà, forse, ai maggiori cambiamenti nel modo di produrre software.

Importanti sviluppi si hanno anche nella progettazione grafica delle interfacce utente (Graphical User Interface, GUI). L'usabilità del software si evolve anche grazie ai risultati delle ricerche di una disciplina collaterale all'Informatica, l'interazione uomo-macchina (Human-Computer Interaction, HCI).

1990 - oggi

- L'impulso maggiore all'evoluzione dell'Ingegneria del software è fornito, senza dubbio, dallo sviluppo delle tecnologie della Rete. Internet è, a tutti gli effetti, una delle maggiori rivoluzioni tecnologiche e culturali del nuovo millennio.

Il crollo dei costi dell'hardware, la grande competitività tra le aziende software e l'uso del Web inducono l'Ingegneria del software a ridefinire i suoi principi: tecnologie, metodologie e culture.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Internet e Open source rappresentano due pilastri imprescindibili che costringono tutti i produttori a ridefinire le proprie strategie. E lo sviluppo del software non può esimersi dall'adeguarsi a tali nuovi paradigmi.

Tra le "nuove" metodologie di sviluppo software spiccano quelle Agili (vedi Manifesto). La loro diffusione è sempre maggiore e spinge modelli divenuti standard di mercato ad uniformarsi a loro (es.: CMMI, PMBOK, TenStep, ecc.). Tra queste, la metodologia SCRUM registra un interesse maggiore.

Attualmente (come pure negli ultimi decenni) possiamo dedurre che delle tre linee di ricerca (Organizzazione, Metodologie e Tecnologie) solo la terza è conosciuta in dettaglio dagli sviluppatori software.

La conoscenza dei linguaggi di programmazione, dei sistemi operativi e di quelli di gestione è tuttora considerata la sola necessaria e sufficiente competenza richiesta per sviluppare software (sarebbe più corretto dire "programmare/produrre codice").

"Sviluppare software" è ben più complesso e richiede anche competenze metodologiche relative all'ingegnerizzazione dei requisiti, alla progettazione del software, al test e collaudo e alle discipline laterali.

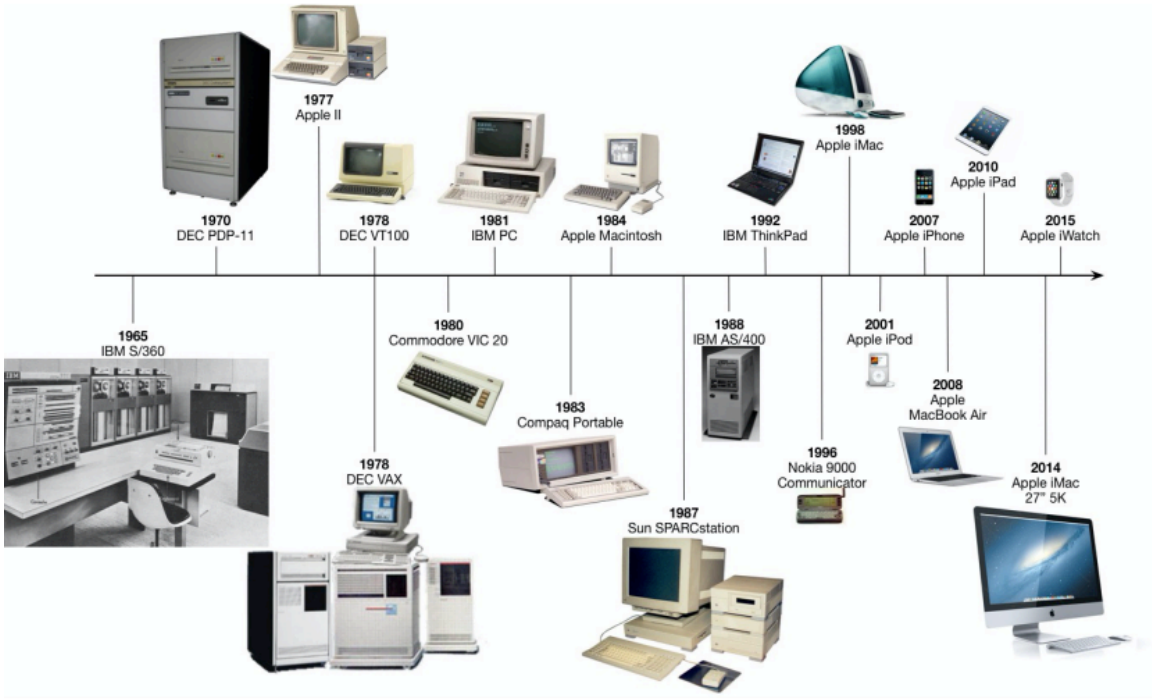
SWEBOK rappresenta una guida ufficiale allo sviluppo delle conoscenze e competenze dell'Ingegneria del software.

Possiamo concludere, forse, che la "crisi del software" non è stata mai superata?

Come ebbe a scrivere Tom DeMarco nel 2009, "L'Ingegneria del software è forse un'idea il cui tempo è venuto e passato?"

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



1965-2015: milestone dell'ingegneria informatica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Miti e considerazioni sullo sviluppo e progettazione del software

Questa sezione ha l'obiettivo di sfatare alcuni miti e considerazioni che spesso vengono riportate quando si parla di sviluppo e progettazione del software. Questi miti sono nati agli inizi degli anni '70, ed alcuni di essi potrebbero essere ancora validi.

Miti del management

Mito: abbiamo già interi volumi di standard e procedure da seguire nello sviluppo. Non c'è forse tutto l'indispensabile?

Realtà: Quanti di questi standard e procedure sono conosciute ed applicate? Sono seguiti in maniera completa ed ottimizzata? Sono usati standard e procedure che riflettano i metodi moderni di sviluppo del software? La risposta a tutte queste domande è NESSUNA.

Mito: Abbiamo i più moderni strumenti di sviluppo. Dopo tutto acquistiamo sempre i computer più recenti.

Realtà: Non è vero che al computer più potente corrisponde un miglior sviluppo del software. È come dare una formula 1 ad uno che deve imparare a guidare... Spesso sarebbe più conveniente spendere i soldi in strumenti adatti allo sviluppo, piuttosto che potenziare i computer, ma costringe- re i programmatori a lavorare a mano (per mancanza di strumenti).

Mito: se siamo in ritardo possiamo recuperare aumentando il numero di programmatori.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Realtà: Sviluppare un software non è, come abbiamo detto, costruire un oggetto. Immettere "un'or- da mongola" all'interno del processo di sviluppo rallenterà ancora di più lo sviluppo, in quanto la nuova gente dovrà essere addestrata, dovrà capire cosa hanno fatto gli altri programmatori, ecc.

Mito: se decido di far realizzare un progetto software ad una terza parte, posso stare tranquillo per- ché tutto il lavoro verrà svolto esternamente.

Realtà: Le "terze parti" non hanno, spesso, idea del funzionamento interno; se non si sa gestire i propri processi interni, è assai difficile riuscire a gestire il processo svolto da uno esterno.

Miti del Cliente

Mito: un'affermazione generica degli scopi è sufficiente per cominciare a scrivere i programmi; i dettagli si possono trattare in seguito.

Realtà: un'insufficiente descrizione preliminare è la causa principale del fallimento del progetto. Senza un'accurata descrizione, non si riesce a definire i confini del progetto.

Mito: I requisiti di un progetto mutano di continuo, ma i mutamenti si gestiscono agevolmente grazie alla flessibilità del software.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Realtà: Il software non è flessibile, né tanto meno lo è svilupparlo. È vero, i requisiti cambiano, ma apportare il cambiamento durante le prime fasi del progetto è più facile e meno costoso, che non quando il software è pronto. Su questo argomento avremo modo di tornare per un approfondimento.

Miti del programmatore

Mito: Una volta scritto e messo in opera un programma, il nostro lavoro è finito. Realtà: La maggior parte del lavoro avviene una volta che il software è stato rilasciato.

Mito: Fino a quando il programma non è in condizioni di essere eseguito, non c'è modo di valutarne la qualità.

Realtà: Uno dei metodi più efficaci di esame della qualità del software, la revisione tecnica formale, si applica già dall'inizio di un progetto. Avremo modo di approfondire l'argomento.

Mito: il solo prodotto di un progetto concluso è il programma funzionante.

Realtà: un programma funzionante è solo una parte del software. La documentazione è il fondamento di uno sviluppo riuscito e, cosa ancora più importante, è la guida dell'attività di manutenzione.

Mito: l'ingegneria del software ci farà scrivere un'inutile e voluminosa documentazione che inevitabilmente rallenterà le cose.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Realtà: l'ingegneria del software punta sulla qualità, la migliore quantità porta ad un minor lavoro, un minor lavoro porta a tempi di consegna più rapidi. Quello che porta via tempo è imparare l'ingegneria del software (insomma, deve essere un investimento). Quando si pensa che non vi è tempo per svolgere un'ingegnerizzazione del software, ci si provi a chiedere: "Avremo tempo di rifarlo da zero?".

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Ghezzi, Jazayeri, Mandrioli, "Ingegneria del software", Paerson.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).